

Sources Ledger – External Quality Research

Field	Value
Revision	2
Created	2026-07-22
Last modified	2026-07-22T15:03:22Z
Status	COMPLETE — 76 sources across 7 rounds
Companion	EXTERNAL_RESEARCH.md

Every source used in `EXTERNAL_RESEARCH.md`, with URL + access date (§11.4.99). Evidence class: `[PR]` peer-reviewed/measured · `[IND]` industrial report with data · `[DOC]` standard or official project documentation · `[ANEC]` practitioner anecdote/opinion.

Round – (accessed –)

- **S1** `[PR]` Xia et al., "An empirical analysis of reopened bugs based on open source projects", EASE 2016. <https://dl.acm.org/doi/10.1145/2915970.2915986>
- **S2** `[PR]` Same study, author copy. https://www.researchgate.net/publication/303515628_An_empirical_analysis_of_reopened_bugs_based_on_open_source_projects
- **S3** `[PR]` Shihab et al., "Predicting Re-opened Bugs: A Case Study on the Eclipse Project", WCRE 2010. https://sailresearch.github.io/sail-website/data/pdfs/WCRE2010_PredictingRe-openedBugs_ACaseStudyOnTheEclipseProject.pdf

- **S4** [PR] Zimmermann, Nagappan, Guo, Murphy, "Characterizing and Predicting Which Bugs Get Reopened", ICSE 2012 (Microsoft Windows). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/zimmermann-icse-2012.pdf>
- **S5** [PR] "Revisiting reopened bugs in open source software systems", Empirical Software Engineering, 2022. <https://link.springer.com/article/10.1007/s10664-022-10133-6>
- **S6** [PR] Inozemtseva & Holmes, "Coverage Is Not Strongly Correlated with Test Suite Effectiveness", ICSE 2014. https://www.cs.ubc.ca/~rtholmes/papers/icse_2014_inozemtseva.pdf (ACM: <https://dl.acm.org/doi/10.1145/2568225.2568271>)
- **S7** [ANEC] Summary + influence note (ICSE 2024 MIP award). <https://neverworkintheory.org/2021/09/24/coverage-is-not-strongly-correlated-with-test-suite-effectiveness.html>
- **S8** [PR] Petrović, Ivanković, Fraser, Just, "Does mutation testing improve testing practices?", ICSE 2021. https://homes.cs.washington.edu/~rjust/publ/mutation_testing_practices_icse_2021.pdf
- **S9** [PR] Petrović et al., "Practical Mutation Testing at Scale: A view from Google", IEEE TSE 2022. https://homes.cs.washington.edu/~rjust/publ/practical_mutation_testing_tse_2021.pdf
- **S10** [PR] Luo, Hariri, Eloussi, Marinov, "An Empirical Analysis of Flaky Tests", FSE 2014. https://www.researchgate.net/publication/301428664_An_empirical_analysis_of_flaky_tests
- **S11** [IND] Google flaky-test mitigation summary (16% figure; 1-in-7). <https://talent500.com/blog/google-flaky-test-mitigation-strategies/> (secondary; primary is Google Testing Blog "Flaky Tests at Google and How We Mitigate Them", J. Micco, 2016)
- **S12** [PR] Parry, Kapfhammer, Hilton, McMinn, "A Survey of Flaky Tests", 2021. <https://philmcminn.com/publications/parry2021.pdf>
- **S13** [PR] "Test flakiness' causes, detection, impact and responses: A multivocal review", JSS 2023. <https://www.sciencedirect.com/science/article/pii/S0164121223002327>
- **S14** [PR-critique] Bossavit, *The Leprechauns of Software Engineering* (Leanpub). <https://leanpub.com/leprechauns/read>

- **S15** [IND] The Register, "Everyone cites that 'bugs are 100x more expensive to fix in production' research, but the study might not even exist", 2021-07-22. https://www.theregister.com/2021/07/22/bugs_expense_bs/
- **S16** [IND] DevStats, "Escaped Defects, Explained: Benchmarks" (Capers Jones DRE ~85%). <https://www.devstats.com/glossary/escaped-defects>
- **S17** [IND] DORA metrics benchmark guides (elite CFR 0–15%). <https://getoptimal.ai/blog/dora-metrics-guide> ; <https://plandek.com/blog/escaped-defects>

2 0 2 6 0 7 2 2

Round (accessed – –)

- **S18** [PR] Barr, Harman, McMinn, Shahbaz, Yoo, "The Oracle Problem in Software Testing: A Survey", IEEE TSE 41(5), 2015. <https://eecs481.org/readings/testoracles.pdf> (ACM: <https://dl.acm.org/doi/10.1109/TSE.2014.2372785>)
- **S19** [PR] Zhang & Mesbah, "Assertions Are Strongly Correlated with Test Suite Effectiveness", ESEC/FSE 2015. <https://people.ece.ubc.ca/amesbah/resources/papers/fse15.pdf> (ACM: <https://dl.acm.org/doi/10.1145/2786805.2786858>)
- **S20** [PR] "Assertions in software testing: survey, landscape, and trends", STTT 2025. <https://link.springer.com/article/10.1007/s10009-025-00794-1>
- **S21** [PR] "Characterizing High-Quality Test Methods: A First Empirical Study", 2022. <https://arxiv.org/pdf/2203.12085>
- **S22** [PR] "Understanding the Prevalence of Test Smells in Open-source and Closed-source Projects", QUASOQ 2024. <https://ceur-ws.org/Vol-3864/quasq-2024-paper-03.pdf>
- **S23** [PR] "On the Diffusion of Test Smells in LLM-Generated Unit Tests", 2024. <https://arxiv.org/html/2410.10628>
- **S24** [PR] "A Survey of Modern Compiler Fuzzing" (EMI/metamorphic bug counts in GCC/LLVM), 2023. <https://arxiv.org/pdf/2306.06884>
- **S25** [PR] Windsor et al., "High-coverage metamorphic testing of concurrency support in C compilers" (C4), STVR 2022. <https://onlinelibrary.wiley.com/doi/full/10.1002/stvr.1812>

- **S26** [PR] Segura et al., "A Survey on Metamorphic Testing", IEEE TSE 2016. <https://eprints.whiterose.ac.uk/110335/1/segura16-tse.pdf>
- **S27** [PR] "An Empirical Evaluation of Property-Based Testing in Python", OOPSLA 2025 (~50× mutant kill per PBT test). https://cseweb.ucsd.edu/~mcoblenz/assets/pdf/OOPSLA_2025_PBT.pdf (ACM: <https://dl.acm.org/doi/10.1145/3764068>)
- **S28** [PR] "Property-Based Testing in Practice", ICSE 2024 (industrial adoption). <https://dl.acm.org/doi/10.1145/3597503.3639581>
- **S29** [ANEC] Chaos engineering practice accounts (Netflix Chaos Monkey lineage; Gremlin guide). <https://www.gremlin.com/chaos-monkey> ; <https://sdtimes.com/softwaredev/how-tech-giants-like-netflix-built-resilient-systems-with-chaos-engineering/>
- **S30** [PR] "Maximizing Error Injection Realism for Chaos Engineering with System Calls", 2020. <https://arxiv.org/pdf/2006.04444>

Round (accessed

- **S31** [DOC] GitHub Docs, "About status checks" / "Troubleshooting required status checks" (skipped = Success semantics). <https://docs.github.com/articles/about-status-checks> ; <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/collaborating-on-repositories-with-code-quality-features/troubleshooting-required-status-checks>
- **S32** [IND] Emmer, "Skippable GitHub Status Checks Aren't Really Required". <https://emmer.dev/blog/skippable-github-status-checks-aren-t-really-required/>
- **S33** [DOC] poseidon/wait-for-status-checks ("require triggered checks pass"). <https://github.com/poseidon/wait-for-status-checks>
- **S34** [DOC] LDRA, "DO-178C demystified" technical briefing (evidence categories, DAL-scaled independence). <https://ldra.com/wp-content/uploads/Ldra/DO-178C-Technical-Briefing-v2.1.pdf>
- **S35** [DOC] Rapita Systems, DO-178C testing (structural coverage, traceability as lifecycle data). <https://www.rapitasystems.com/do178c-testing>
- **S36** [DOC] ISO 26262 Academy, "Confirmation Measures". <https://iso26262.academy/features/concepts/confirmation-measures>

- **S37** [DOC] Delphi/NMI, "Tier-1 perspective on ISO 26262 Confirmation Measures" (independence in practice). https://nmi.org.uk/wp-content/uploads/2016/06/3_Tier1-Perspective-on-challenges-of-functional-safety-DELPHI.pdf
- **S38** [PR] SEI blog, "Eliminative Argumentation: A Means for Assuring Confidence in Safety-Critical Systems". <https://insights.sei.cmu.edu/blog/eliminative-argumentation-a-means-for-assuring-confidence-in-safety-critical-systems/>
- **S39** [PR] Goodenough, Weinstock, Klein, "Eliminative Argumentation: A Basis for Arguing Confidence in System Properties". https://www.researchgate.net/publication/272678149_Eliminative_Argumentation_A_Basis_for_Arguing_Confidence_in_System_Properties
- **S40** [PR] "A Taxonomy of Real-World Defeaters in Safety Assurance Cases", 2025 + CoDefeater (2024). <https://arxiv.org/html/2502.00238> ; <https://arxiv.org/pdf/2407.13717>

Round – (accessed –)

- **S41** [DOC] SQLite, "How SQLite Is Tested" (100% MC/DC; harness independence; fuzzing-vs-MC/DC statement). <https://sqlite.org/testing.html>
- **S42** [DOC] SQLite, "TH3" harness documentation. <https://sqlite.org/th3.html>
- **S43** [IND] Zalewski (lcamtuf), "Finding bugs in SQLite, the easy way" (AFL, 22 crashing cases, 2015). <https://lcamtuf.blogspot.com/2015/04/finding-bugs-in-sqlite-easy-way.html>
- **S44** [DOC] Linux kernel docs, "Handling regressions" (the "first rule"). <https://docs.kernel.org/process/handling-regressions.html>
- **S45** [DOC] Leemhuis, Linux kernel regression tracking / regzbot. <https://linux-regtracking.leemhuis.info/about/>
- **S46** [IND] KernelCI Foundation, "Regzbot Joins KernelCI" (2026-05-04; release decisions consult tracking). <https://kernelci.org/blog/2026/05/04/regzbot-joins-kernelci-strengthening-linux-kernel-regression-tracking/>
- **S47** [IND] Stenberg, daniel.haxx.se testing tag (1,900+ tests, ~130 environments, ~140k executions/change). <https://daniel.haxx.se/blog/tag/testing/>

- **S48** [DOC] Chromium infra, Flake Portal documentation. https://chromium.googlesource.com/infra/infra/+03bc79323fb18816a0b5573d83cc53ac48d92235/appengine/findit/docs/flake_portal.md
- **S49** [DOC] Chromium OS, "Breakage and Flake Policy". <https://www.chromium.org/chromium-os/developer-library/guides/testing/breakages-and-flakes/>
- **S50** [DOC] Chromium, "Sheriffing" documentation. <https://chromium.googlesource.com/chromium/src/+80.0.3987.87/docs/sheriff.md>
- **S51** [PR] Duplicate bug-report corpus studies (Mozilla $\leq 30\%$, Eclipse $\sim 17\text{--}20\%$; 2017–2022 replication 6.6–20.5%). <https://smilevo.github.io/DupBugRep-Scripts/> ; <https://greg4cr.github.io/pdf/24duplicates.pdf>
- **S52** [PR] "Towards Understanding the Impacts of Textual Dissimilarity on Duplicate Bug Report Detection" ($\leq 23\%$ textually dissimilar duplicates), 2022. <https://arxiv.org/pdf/2212.09976>

Round (accessed)

- **S53** [PR] Dong et al., "Bash in the Wild: Language Usage, Code Smells, and Bugs", ACM TOSEM 2022. <https://dl.acm.org/doi/10.1145/3517193> (author copy: <https://yiwendong.com/assets/pdf/tosem22.pdf>)
- **S54** [DOC] Woledge, BashFAQ/105 (why `set -e` is unpredictable). <https://mywiki.woledge.org/BashFAQ/105>
- **S55** [IND] Oil shell, "Can Unix Shell Error Handling Be Fixed Once and For All?" (strict-mode holes incl. SIGPIPE-under-pipefail). <https://www.oilshell.org/blog/2022/05/release-0.10.0.html>
- **S56** [DOC] `ps | grep self-match canon + workarounds`. <https://www.baeldung.com/linux/grep-exclude-ps-results> ; <https://docs.aws.amazon.com/codeguru/detector-library/shell/ps-grep-alternative>
- **S57** [PR] Johnson, Song, Murphy-Hill, Bowdidge, "Why Don't Software Developers Use Static Analysis Tools to Find Bugs?", ICSE 2013. <https://cs.gmu.edu/~johnsonb/docs/icse2013.pdf>

- **S58** [PR] Sadowski et al., "Lessons from Building Static Analysis Tools at Google", CACM 61(4), 2018 (<10% effective-FP bar). <https://cacm.acm.org/research/lessons-from-building-static-analysis-tools-at-google/>
- **S59** [DOC] SWE Book ch.20, "Tricorder: Google's Static Analysis Platform". <https://abseil.io/resources/swe-book/html/ch20.html>
- **S60** [PR] Lipsitch, Tchetgen Tchetgen, Cohen, "Negative controls: a tool for detecting confounding and bias in observational studies", Epidemiology 2010. <https://pubmed.ncbi.nlm.nih.gov/20335814/>
- **S61** [PR] "A Selective Review of Negative Control Methods in Epidemiology", 2021; "Advances in methodologies of negative controls: a scoping review", J Clin Epi 2023. <https://arxiv.org/pdf/2009.05641> ; [https://www.jclinepi.com/article/S0895-4356\(23\)00318-9/fulltext](https://www.jclinepi.com/article/S0895-4356(23)00318-9/fulltext)
- **S62** [PR] Sanderson et al., "Negative control exposure studies in the presence of measurement error" (validity condition: shared error sources). <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5913619/>

2 0 2 6 1 0 7 2 2

Round (accessed – –)

- **S63** [PR] Keystone ICU project account (Pronovost; central-line infections). <https://hsph.harvard.edu/news/fall08checklist/>
- **S64** [PR] Haynes, Gawande et al., "A Surgical Safety Checklist to Reduce Morbidity and Mortality in a Global Population", NEJM 2009. <https://www.nejm.org/doi/full/10.1056/NEJMsa0810119>
- **S65** [PR] Urbach et al., "Introduction of Surgical Safety Checklists in Ontario, Canada", NEJM 2014 (null result across 101 hospitals). <https://www.nejm.org/doi/full/10.1056/NEJMsa1308261>
- **S66** [DOC] AHRQ PSNet, "Checklists" primer (reconciling positive vs null results: implementation depth). <https://psnet.ahrq.gov/primers/primer/14/Checklists>
- **S67** [DOC] Hollnagel, "From Safety-I to Safety-II: A White Paper". <https://www.england.nhs.uk/signuptosafety/wp-content/uploads/sites/16/2015/10/safety-1-safety-2-white-papr.pdf>

- **S68** [DOC] EU-OSHA OSHwiki, "Violation of OSH rules and procedures" (routine violations become invisible). <https://oshwiki.osha.europa.eu/en/themes/violation-osh-rules-and-procedures>
- **S69** [PR] "Aircrews, Rules and the Bogeyman: Mapping the Benefits and Fears of Noncompliance", Safety 2023. <https://www.mdpi.com/2313-576X/9/1/15/xml>
- **S70** [PR] Felisberto et al., "Override rate of drug-drug interaction alerts in CDSS: systematic review and meta-analysis" (~90%, CI 85–95%), Health Informatics J 2024. <https://journals.sagepub.com/doi/10.1177/14604582241263242>
- **S71** [PR] "Appropriateness of Overridden Alerts in CPOE: Systematic Review", JMIR Med Inform 2020. <https://medinform.jmir.org/2020/7/e15653>
- **S72** [PR] "Predicting employee information security policy compliance on a daily basis: security-related stress, emotions, and neutralization", Inf. & Mgmt 2019. <https://www.sciencedirect.com/science/article/abs/pii/S0378720618300739>
- **S73** [IND] UAlbany 2026, "'Security fatigue' may weaken digital defenses". <https://www.albany.edu/news-center/news/2026-study-security-fatigue-may-weaken-digital-defenses>

2 0 2 6 0 7 2 2

Round (accessed – –)

- **S74** [PR] Yuan et al., "Simple Testing Can Prevent Most Critical Failures", OSDI 2014. <https://www.usenix.org/system/files/conference/osdi14/osdi14-paper-yuan.pdf> (project page: <https://www.eecg.toronto.edu/failureAnalysis/>)
- **S75** [PR] Eder, Hauptmann, Junker, Juergens, Vaas, Prommer, "Did We Test Our Changes?" (untested changes ~5× defect-prone; ~50% of changes shipped untested). Summarised in Teamscale/CQSE Test Gap Analysis documentation. <https://docs.teamscale.com/reference/test-gap-analysis/> ; <https://www.cqse.eu/en/news/blog/bridge-your-test-gaps-with-teamscale/>
- **S76** [PR] "Prioritizing Test Gaps by Risk in Industrial Practice", 2025. <https://www.se.cs.uni-saarland.de/publications/docs/HSJ+25.pdf>